

START HERE

From paper proof to program

A guided route through BFS, insertion sort, and the proof obligations you actually write.

You already know what a loop invariant is. This tutorial shows how to turn that knowledge into a checked correctness and time theorem for a RAM program. **No compiler knowledge is assumed.**

RUNNABLE LEAN

```
import AlgoLib.Experimental.RAM
open AlgoLib.Experimental.RAM.Paper

#eval (Insertion.run [3, 1, 2]).value
-- [1, 2, 3]
```

Your destination

Run an algorithm, explain its invariant and charging argument, and identify which obligations are yours and which the library discharges.

Choose a route

I want to ...	Read
Run an example and understand its output	Pages 2-4
Understand correctness and time credits	Pages 5-7
Read and interact with a real Lean proof	Pages 8-9
Follow each component with a side-by-side demo	Pages 10-19
Troubleshoot proofs and solve exercises	Pages 20-21

Before you begin

Use a checkout of **experimental-RAM-model** with Lean and Lake available. The repository pins Lean in lean-toolchain; use that version. The companion **StudentDemo.lean** is a complete, checked file. Code excerpts below live inside its namespace unless a box says otherwise.

Labels distinguish **RUNNABLE LEAN**, **REPOSITORY EXCERPT**, and **PSEUDOCODE**. You can understand the paper argument before learning every Lean tactic.

1. GET A FIRST RESULT

Run a graph you can draw

The demo has a diamond-shaped component and one isolated vertex.

RUNNABLE LEAN / INSIDE STUDENTDEMO.LEAN

```
def graph : EdgeInput where
  n := 5
  edges := [(0, 0, 1), (1, 0, 2),
            (2, 1, 3), (3, 2, 3)]
  distinct := by decide
  valid := by decide

#eval (BFS.run (graph.fromSource 0 (by decide))).value.toList
-- [0, 1, 2, 3]

#eval (BFS.run (graph.fromSource 4 (by decide))).value.toList
-- [4]
```

Each triple is **(edge label, endpoint, endpoint)**. Vertex identifiers are 0 through 4. `distinct` checks the finite edge data; `valid` checks that the endpoints are in range. The source proof checks that the chosen source is a vertex.

Run the companion

Copy **StudentDemo.lean** into the repository root. From that directory, run:

SHELL COMMANDS

```
lake build
lake env lean StudentDemo.lean
```

The file uses `#guard_msgs` to check expected outputs. A successful run can therefore be silent. In your editor, select a `#eval` command to inspect its result, or remove its surrounding output assertion while experimenting.

Read the result correctly

value is a visited bitmap; **toList** displays its marked vertices in increasing order. It is not the dequeue order. **steps** counts actual compiled RAM steps. You supply no fuel.

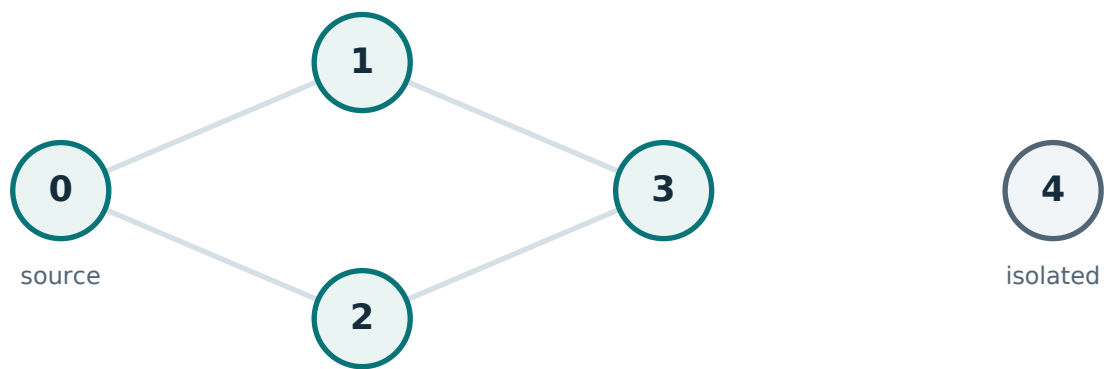
[Get the complete StudentDemo.lean companion](#)

[Open the checked example API](#)

2. MAKE THE STATE CONCRETE

Trace the frontier by hand

At each outer-loop boundary, all neighbors of the processed vertices have been examined.



Use source 0 and list its neighbors as [1, 2]. FIFO means that newly discovered vertices join the back of the queue. Vertex 3 is discovered from vertex 1; scanning vertex 2 must not enqueue it again.

After	processed	queue	seen
Initialization	{ }	[0]	{0}
Process 0	{0}	[1, 2]	{0, 1, 2}
Process 1	{0, 1}	[2, 3]	{0, 1, 2, 3}
Process 2	{0, 1, 2}	[3]	{0, 1, 2, 3}
Process 3	{0, 1, 2, 3}	[]	{0, 1, 2, 3}

Check your understanding

Why is 3 present only once in the queue? Because discovery marks it **before** enqueueing it. The later edge from 2 sees that mark. Vertex 4 is never discovered.

The displayed state is mathematical. The library represents it with visited flags, a FIFO, and an adjacency cursor in RAM. You reason about the table; the library proves that the representation follows it.

3. CONNECT PSEUDOCODE TO THE API

Read the program in layers

The current interface uses certified operations. Some textbook lines are grouped into one library call.

PSEUDOCODE / FOR READING, NOT LEAN INPUT

```
clear all visited flags; mark and enqueue source
while Q is not empty:
  u := dequeue(Q)
  for v in adjacency[u]:
    if not visited[v]:
      visited[v] := true
      enqueue(Q, v)
```

REPOSITORY EXCERPT / PAPER/BFS.LEAN

```
def program (a : Adjacency) : Program (model a) := paper {
  while (queueNonempty a) {
    call dequeue a;
    call (scanNeighbors a).call;
    call finish a;
  }
}
```

Component	Meaning you use
Input preparation	Clears flags, marks the source, and seeds the FIFO. This is compiled and charged.
dequeue	Removes the front vertex and opens that vertex's adjacency row.
scanNeighbors	Scans the row; marks and enqueues previously unseen vertices. Its loop has a reusable proof.
finish	Records the current vertex in the mathematical processed set. It emits no instructions.

What “call” promises: if you establish the precondition, you may use the operation's mathematical effect and cost bound. Its implementation proof already includes the physical memory frame.

[Read the row procedure and its contracts](#)

4. STATE THE ALGORITHMIC REASON

Write the invariant in English

An invariant describes every loop boundary, not just the final answer.

Invariant fact	Why the proof needs it
seen = processed $\cup$ queued	Every discovered vertex is either finished or still waiting.
No duplicates; processed and queue are disjoint	Each vertex is processed at most once.
The source is seen; every seen vertex is reachable	No unrelated vertex enters the answer.
All neighbors of processed vertices are seen	When the queue empties, no reachable vertex can have been missed.
No partially scanned row at the outer header	Each outer iteration completely finishes one vertex.

The paper argument

Initialization. Only the source is seen and queued; processed is empty. Reachability of the source follows from the length-zero path. Closure for processed vertices is vacuous.

Preservation. Dequeue a reachable vertex u . Every newly discovered neighbor is reachable via u . Mark-before-enqueue preserves uniqueness. After scanning the whole row, u may safely join processed.

Exit. The queue is empty, so seen = processed. The set contains the source and is closed under following edges. Induction along any path from the source shows that every reachable vertex is seen. The soundness part gives the reverse direction.

Two different conclusions

Reachability: marked(v) iff v is reachable from the source, even on a disconnected graph.

Connectivity: every graph vertex is marked iff the graph is connected, given a valid source.

[Read the packaged graph invariant](#)

5. MEET SYMBOLIC EXECUTION

See what a proof goal asks

A verification condition is a list of facts needed to justify the next command and the desired result.

For an operation with precondition P , effect E , and cost c , symbolic execution asks you to prove: **P holds; c fits the budget; the desired postcondition holds after E .** It substitutes the logical effect instead of exposing RAM instructions.

RUNNABLE LEAN / LOCAL CONTRACT DEMO

```
example (s : Insertion.State) (safe : s.todo ≠ []) :
  VC (.action Insertion.insertNext)
    (fun t _ => t = Insertion.effect s)
    s (s.sorted.length + 1) := by
  paper_steps []
  exact (safe, Nat.le_refl _, by trivial)
```

What you see	How to read it
<code>safe : s.todo ≠ []</code>	The insertion operation has a value available to insert.
<code>fun t _ => ...</code>	The desired postcondition describes the new state <code>t</code> . The underscore ignores leftover credits.
<code>s.sorted.length + 1</code>	The supplied budget, measured in the library's work units.
<code>paper_steps []</code>	Use registered operation contracts to generate logical obligations.
<code>Nat.le_refl _</code>	The supplied budget equals the required cost, so it fits.

Try it in the editor

Put the cursor after `paper_steps []` and inspect the goals. Temporarily replace the budget by 0. The proof should fail because a nonempty insertion costs positive work. Restore the original budget afterward.

This is a warm-up about one operation. A loop additionally needs an invariant, a potential, and an exit argument. You supply those mathematical facts; the framework propagates the contracts.

6. PROVE A LINEAR BOUND

Pay once for each vertex

Credits are a proof budget. They do not exist in the executed data structure.

Let d be the length of the dequeued vertex's adjacency row. The public contracts allocate the following work units for one outer iteration:

Work	Credits
True outer-loop guard	1
dequeue and open the row	1
scanNeighbors: d visits and $d + 1$ row tests	$2d + 1$
finish: ghost bookkeeping	0
Total per processed vertex	$3 + 2d$

Choose a potential

$\Phi = \text{sum of } (3 + 2 \text{ degree}(v)) \text{ over all unprocessed vertices.}$

A fresh vertex leaves this sum when it is processed. The removed summand pays for its whole iteration.

On the diamond-plus-isolated-vertex demo, the four diamond vertices each have degree 2. They receive 7 credits each; vertex 4 receives 3. The potential follows $31 \rightarrow 24 \rightarrow 17 \rightarrow 10 \rightarrow 3$. The unspent 3 belongs to the unreachable vertex. Reserve one extra credit for the final false outer guard.

In general, the initial potential is $3n + 2A$, where A is the number of stored adjacency entries. For an undirected graph, $A \leq 2m$. Thus the loop uses at most $3n + 4m + 1$ work units. Certified input preparation also takes linear work.

The executable theorem

BFS.linear input proves $\text{steps} \leq 370(n + m)$ for actual compiled RAM instructions, including initialization. The framework handles the conversion from library work units. You do not derive the compiler constant.

The bound is conservative, not an exact runtime. Here $n = 5$ and $m = 4$, so it gives $\text{steps} \leq 3330$. Host input encoding and output formatting are outside the RAM count; this is unit-cost RAM complexity, not bit complexity.

7. A SECOND INVARIANT

Use insertion sort to practise

This implementation grows a sorted suffix from right to left.

REPOSITORY EXCERPT / PAPER/INSERTIONSORT.LEAN

```
def program : Program model := paper {
  while (more) { call insertNext; }
}

def invariant (input : List Nat) (s : State) : Prop :=
  s.sorted.Pairwise ( $\cdot \leq \cdot$ )  $\wedge$  (s.todo ++ s.sorted).Perm input

def potential (s : State) : Nat :=
  s.todo.length * (s.todo.length + s.sorted.length + 2)
```

todo lists the unprocessed prefix in processing order, so it starts as the reverse of the input. **sorted** is the sorted suffix. `insertNext` inserts the head of `todo` into that suffix and removes it from `todo`.

Input [3, 1, 2]	todo	sorted	Potential
Initially	[2, 1, 3]	[]	15
Insert 2	[1, 3]	[2]	10
Insert 1	[3]	[1, 2]	5
Insert 3	[]	[1, 2, 3]	0

Pairwise ($\cdot \leq \cdot$) means that earlier elements are no greater than later ones. **Perm** means that the same values occur with the same multiplicities. Both matter: a sorted output that loses a duplicate is not a correct sort.

Each iteration removes one `todo` value, preserves the total length, and uses at most one suffix scan plus guard overhead. The potential drops by $n + 2$, enough to pay. At exit `todo` is empty, so the sorted suffix is a permutation of the entire input.

What is supplied here

The library already proves the insertion operation's mathematical effect and linear cost. This exercise teaches how to verify a client loop. Implementing the insertion primitive itself is a separate library task.

8. INTERACT WITH THE THREE OBLIGATIONS

Read a complete loop proof

Open Paper/InsertionSort.lean alongside this page. The proof is complete; the boxes below explain its structure.

PROOF OUTLINE / NOT A COMPLETE LEAN PROOF

```
theorem loopProof (input : List Nat) : LoopProof
  more (.action insertNext) (invariant input) potential
  (fun s => s.todo = [] ∧
    s.sorted.Pairwise (· ≤ ·) ∧ s.sorted.Perm input) where
  preservation := by
    -- Use the insertion contract and list lemmas.
    -- Show the invariant and remaining budget at the new state.
  payment := by
    -- A true guard has at least one available credit.
  exit := by
    -- Empty todo makes the invariant imply the result.
```

Preservation. The real proof splits todo into empty or $x :: xs$. The true guard excludes the empty case. It uses a sorted-insertion lemma, a permutation lemma, and `paper_steps`. Credit arithmetic is discharged by `paper_credits`.

Payment. A nonempty todo gives a positive potential. Paying for the true guard ensures strict progress in the remaining budget, so the same argument establishes termination.

RUNNABLE LEAN / THE EXIT ARGUMENT

```
example (xs : List Nat) (s : Insertion.State)
  (h : Insertion.invariant xs s) (empty : s.todo = []) :
  s.sorted.Pairwise (· ≤ ·) ∧ s.sorted.Perm xs := by
  simpa [Insertion.invariant, empty] using h
```

Read this as: “Expand the invariant, substitute that todo is empty, and simplify.” This is the paper-level exit argument written in Lean. The completed `loopProof.correct` theorem supplies total correctness; the executable interface combines it with initialization and runs the compiler.

Keep the important boundary

You must still prove that the prepared input satisfies the invariant. The library handles memory preparation and its cost; it cannot decide that your chosen invariant is initially true.

[Open the full, checked loop proof](#)

9. KEEP THIS MAP BESIDE YOUR EDITOR

How the components connect

The map is the overview. Pages 11-19 expand every connection with construction and student use side by side.

YOU**Program + invariant + potential**

Describe the algorithm, what stays true, and how work is paid.

**LIBRARY****Operation and procedure contracts**

Provide logical effects, preconditions, cost bounds, and frames.

**AUTOMATIC****Verification conditions**

Substitute effects and collect obligations; you prove the remaining math.

**AUTOMATIC****Checked source and compiler**

Turn the verified composition into typed source, then RAM instructions.

**AUTOMATIC****Input/output interface and runner**

Include paid initialization, run without fuel, and expose the result.

**YOU USE****Correctness + time theorem**

Apply `run_correct`, `connected_iff`, or the time bound to the same run.

Follow one write. A visit changes a flag and perhaps a queue slot. Its certified contract preserves the graph representation. Your proof sees a set/list update. The VCG uses that update; the compiler theorem connects the resulting proof to the actual writes. You never prove adjacency-pointer preservation in the algorithm proof.

COMPONENT WALKTHROUGH / 1 OF 9

1. Choose the logical state

Start with the values in your paper argument. Reuse a representation that the library already verifies.

LIBRARY CONSTRUCTION

A **Model** relates a mathematical state to the implementation store. Its **Represents** relation explains which memory contents and variables realize that state. **overhead** bounds implementation cost per abstract credit.

```
structure Model (State : Type) where
  Represents : State → Store → Prop
  overhead : Nat
```

STUDENT USE

Reuse **Insertion.State** and **Insertion.model**. The state has two lists: values still to insert and the sorted suffix. Your invariant and postcondition talk about those lists.

```
-- Logical starting state for [9, 4]
Insertion.initial [9, 4]
-- todo = [4, 9], sorted = []
```

The same input, seen at two levels

Paper state: what you use	Representation: what the library connects
todo = [4, 9]; sorted = []	Input array contains [9, 4]. The reverse of todo is the unprocessed array prefix.
todo = [9]; sorted = [4]	The unprocessed prefix is [9]; the suffix holds [4].
todo = []; sorted = [4, 9]	The whole represented array is the sorted suffix.

The logical lists describe the computation. They are **not** the program that runs: the runner executes certified RAM instructions. Evaluating a convenient Lean list function alone would not establish a RAM cost theorem.

What passes to the next step

Each action receives a state satisfying **Represents** and must return a state satisfying the same representation relation after its promised mathematical update.

When would I build a new model? When adding a new memory representation or new primitive data structure. For this demo, reuse the existing one. A new combination of existing operations does not require another representation proof.

[Construction: Model and its operation boundary](#)

[Concrete array representation \(optional maintainer reading\)](#)

COMPONENT WALKTHROUGH / 2 OF 9

2. Use a certified operation

An operation supplies three facts to the algorithm proof: when it is safe, what it changes, and how much work to budget.

LIBRARY CONSTRUCTION

An **Action** bundles requires, effect, work, and an implementation. Its **correct** field proves execution realizes the effect and costs at most $\text{overhead} \times \text{work}$. A cost annotation needs this proof.

```
-- Public contract equations
requires s ↔ s.todo ≠ []
effect s = Inertion.effect s
work s = s.sorted.length + 1
```

STUDENT USE

For **insertNext**, show that `todo` is nonempty. Use the effect to update the lists, and reserve $\text{sorted.length} + 1$ credits. The low-level insertion proof is already supplied.

```
-- A command in paper { ... }
call Inertion.insertNext;

-- [4, 9], [] → [9], [4]
```

Left: contract sketch. Below: a complete local proof at any logical state `s`.

RUNNABLE LEAN

```
example (s : Inertion.State) (safe : s.todo ≠ []) :
  VC (.action Inertion.insertNext)
    (fun t _ => t = Inertion.effect s)
    s (s.sorted.length + 1) := by
  paper_steps []
  exact {safe, Nat.le_refl _, by trivial}
```

After paper_steps	Why it holds
The call is permitted.	safe supplies <code>todo ≠ []</code> .
The available credits cover its work.	The chosen budget equals the contract work.
The resulting state satisfies the postcondition.	The effect substituted by symbolic execution is exactly the requested effect.

Connection to sequencing

The first operation's effect becomes the next operation's input state. Remaining credits become the next operation's budget. This is the information that VC generation carries forward.

[Stable public insertion equations used by paper_steps](#)

COMPONENT WALKTHROUGH / 3 OF 9

3. Preserve unrelated facts

A visit may change seen flags and the FIFO. It must preserve the graph data and any untouched logical fields.

LIBRARY CONSTRUCTION

ReadsOnly R P says assertion P depends only on cells in R. **WritesOnly W** bounds the cells a command changes. A proof that R and W are disjoint lets the generic frame rule preserve P with the same cost bound.

```
-- Frame rule, mathematical reading
ReadsOnly R P
WritesOnly W before after
Disjoint R W
P before
-----
P after
```

STUDENT USE

At the algorithm layer, use the action's logical update. Fields absent from that update stay equal. The certified implementation already establishes that writes to visited flags and the FIFO preserve the graph representation.

```
-- Logical effect of a visit
seen, queue, row may change
current stays equal
processed stays equal
```

RUNNABLE LEAN

```
example (a : Adjacency) (s : Search.State)
  (safe : (Search.visit a).requires s) :
  VC (.action (Search.visit a))
    (fun t _ => t.current = s.current ∧
      t.processed = s.processed) s 1 := by
  paper_steps [Search.visitEffect]
  exact (safe, by omega, by trivial)
```

This proof expands the **mathematical** visit effect. The record update leaves current and processed untouched, so the requested equalities simplify. No adjacency address or queue address appears in the student proof.

Two related mechanisms

Physical framing is proved using read/write footprints inside library contracts. **Logical framing** here follows by simplifying unchanged record fields. Both support the same user experience, but they are distinct proof steps.

The current library provides a reusable footprint rule; it does not infer arbitrary ownership layouts. A new data-structure implementation must supply its footprints and separation proof once.

[Construction: reusable footprint and contract frame rules](#)

COMPONENT WALKTHROUGH / 4 OF 9

4. Package a small procedure

Verify a useful composition once. Later clients should only need its summary.

LIBRARY CONSTRUCTION

A **Procedure** stores its body, precondition, effect, work bound, and a proof named verification. **Procedure.call** turns it into an Action. The compiler still includes the body; client VC generation uses the summary.

```
-- Procedure fields
body
requires
effect
work
verification
```

STUDENT USE

Write two ordinary calls and name the result. Here the procedure inserts two todo values. The precondition requires at least two values; the suffix length determines its budget.

```
def twoBody :
  Program Insertion.model := paper {
    call Insertion.insertNext;
    call Insertion.insertNext;
  }
```

RUNNABLE LEAN

```
def twoInsertions : Procedure Insertion.model where
  body := twoBody
  requires s := 2 ≤ s.todo.length
  effect s := Insertion.effect (Insertion.effect s)
  work s := 2 * s.sorted.length + 3
  verification := twoBody_correct

def client : Program Insertion.model := paper {
  call twoInsertions.call;
}
```

twoBody_correct is proved on the next page and appears before this definition in StudentDemo.lean. There are no missing proof placeholders in the companion.

Cost calculation. If the initial suffix length is r , the first insertion needs $r + 1$ credits. Its effect increases the suffix length to $r + 1$, so the second needs $r + 2$. Together: **$2r + 3$** .

Same pattern in BFS

scanNeighbors packages a whole adjacency-row loop. A BFS client uses its effect and its bound $2 \times \text{row.length} + 1$ without repeating the row-loop proof.

[Construction: Procedure and Procedure.call](#)

COMPONENT WALKTHROUGH / 5 OF 9

5. Generate and solve the VCs

The generator handles composition. You establish the mathematical facts needed by the resulting obligations.

LIBRARY CONSTRUCTION

VC computes obligations recursively from Program. For a sequence, the second fragment's VC becomes the first fragment's continuation. **VC.correct** proves that satisfying them gives total correctness within the supplied budget.

```
-- Definition, abbreviated reading
VC (seq a b) Q s c
  = VC a (VC b Q) s c
```

STUDENT USE

Apply VC.correct, name the initial state and precondition, and run paper_steps. In this proof a case split exposes the first two todo values. Simplification and arithmetic finish the generated obligations.

```
-- Facts you supply
2 ≤ s.todo.length
-- Then both calls are safe;
-- costs are r + 1 and r + 2.
```

RUNNABLE LEAN

```
theorem twoBody_correct : Correct twoBody
  (fun s => 2 ≤ s.todo.length)
  (fun s t => t = Insertion.effect (Insertion.effect s))
  (fun s => 2 * s.sorted.length + 3) := by
  apply VC.correct
  intro s hs
  cases ht : s.todo with
  | nil => simp [ht] at hs
  | cons x xs =>
    cases xs with
    | nil => simp [ht] at hs
    | cons y ys =>
      paper_steps [twoBody, Insertion.effect, ht]
      exact (by simp, by omega, by simp, by omega, by trivial)
```

Read the final five conjuncts in order: **safe first call; pay first call; safe second call; pay second call; final effect**. omega solves the natural-number arithmetic, including subtraction of spent credits. The stated budget is an upper bound, not a claim that every machine run takes that many steps.

Interact with this proof

Put the editor cursor after **paper_steps** and inspect the goal. Replace $2 \times \text{sorted.length} + 3$ by 0 to see a payment obligation fail. Restore the bound before checking the companion.

[How paper_steps and paper_credits are defined](#)

COMPONENT WALKTHROUGH / 6 OF 9

6. Scale from calls to loops

Replace “insert twice” by “insert while values remain.” The student adds the invariant and charging argument.

LIBRARY CONSTRUCTION

LoopProof asks for preservation, payment, and exit. Preservation must retain the invariant and enough potential after the body. A paid true guard gives strict progress.

LoopProof.correct derives termination and a budget of initial potential + 1.

```
-- Obligations, schematic
preservation: I and credits survive
payment: true guard can be paid
exit: I and stopped imply result
```

STUDENT USE

Reuse the same insertNext contract. Supply the sortedness/permutation invariant and the potential from page 8. The loop proof on page 9 discharges the three obligations. Its invariant is not inferred by the library.

```
-- Actual insertion-sort body
paper {
  while (Insertion.more) {
    call Insertion.insertNext;
  }
}
```

Your paper argument	Library hook
The suffix is sorted and all values are preserved.	invariant: sortedness and permutation.
Insertion preserves the invariant and leaves enough credits.	preservation: insertion lemmas, paper_steps, and paper_credits.
A nonempty todo has positive potential.	payment: pay the true guard.
Empty todo makes the suffix the result.	exit: simplify the invariant.
The prepared input satisfies the invariant.	The initial-condition proof passed to Interface.run.

BFS uses the same structure. Its invariant records the frontier and processed vertices. Its potential pays for each processed vertex and scanned adjacency entry; scanNeighbors supplies the row-level summary. See pages 5 and 7 for the argument in graph terms.

What becomes automatic

Given the completed loop proof, total correctness supplies finite execution and the work bound. There is no extra fuel argument, recursive termination proof, or compiler proof for the student to construct.

[Complete insertion-sort invariant and potential proof](#)

COMPONENT WALKTHROUGH / 7 OF 9

7. Compile the verified program

The library connects mathematical execution to typed commands, then to RAM instructions. Your certificate drives this connection.

LIBRARY CONSTRUCTION

Program.source replaces each action by its certified implementation and preserves the control flow. **Run.refines** proves the represented state and cost bound survive this translation. **Eval.compile** then proves execution of the resulting RAM code.

```
-- Translation, schematic
action a → a.implementation
seq a b → seq a.source b.source
loop q b → loop q.impl b.source
```

STUDENT USE

Supply the checked client proof to the runner on the next page. You do not call a compiler theorem or prove a register relation. The tiny equality below is an optional inspection: procedure packaging has not erased its body.

```
-- Checked in StudentDemo.lean
example :
  client.source = twoBody.source :=
  rfl
```

What each boundary guarantees

Boundary	Guarantee established once by the library
Logical Run → typed Eval	The implementation realizes the same logical result. Its cost is at most <code>model.overhead</code> times the abstract work.
Typed Eval → RAM Exec	The compiled execution has the same typed-store observation and the same cost <code>k</code> used by the typed semantics.
Execution proof → runner	The existence of a terminating execution justifies the fuel-free runner. Its reported steps and final state match that execution.

The cost of a typed command is defined to account for its RAM implementation. Therefore the second boundary is cost preserving; it is not an assumption that every source expression is a single machine step. Constants arising from implementation are already absorbed by the supplied contracts.

Why the proof applies to the code you run

The action certificate refers to its actual implementation. Procedures compile their verified body, and the runner compiles the program named in your certificate. There is no separate unverified replacement program selected for execution.

[Construction: source translation and Run.refines](#)

[Construction: compiler correctness](#)

COMPONENT WALKTHROUGH / 8 OF 9

8. Connect to an executable

Reuse the input/output adapter, then pass it the client certificate. This is the bridge from a proof to a run.

LIBRARY CONSTRUCTION

Interface provides a logical initial state, input encoding, paid preparation, output decoding, and an observation relation. Its proofs connect preparation and observation to the model. Generic refinement and compiler theorems connect the client to RAM execution.

```
-- Adapter fields (grouped)
initial, encode
prepare, preparationCost
preparation
decode, Observes, output
```

STUDENT USE

Choose **Insertion.interface**. For our two-call client the input is $[x, y]$, so its precondition is immediate. The interface returns a **Result** containing the observed value and actual compiled steps.

```
-- Adapter already provided
Insertion.interface

-- Certificate from the next proof
client_correct
```

RUNNABLE LEAN

```
theorem client_correct : Correct client
  (fun s => 2 ≤ s.todo.length)
  (fun s t => t = Insertion.effect (Insertion.effect s))
  (fun s => 2 * s.sorted.length + 3) := by
  apply VC.correct
  intro s hs
  paper_steps [client, twoInsertions]
  exact ⟨hs, Nat.le_refl _, by trivial⟩

def sortTwo (x y : Nat) : Result (List Nat) :=
  Insertion.interface.run client_correct [x, y] (by
    simp [Insertion.interface, Insertion.initial])
```

The client proof reads only the procedure summary. It does not unfold **twoBody**, repeat the two-call proof, or mention a register. The runner uses the verified implementation of the same client whose certificate you supplied.

What the step count covers

The bound covers the prepared-input RAM computation, including the interface's compiled preparation. Host-side input encoding, output observation, Lean proof checking, and wall-clock execution time are outside this RAM count.

[Construction: Interface.run and Interface.correct](#)

COMPONENT WALKTHROUGH / 9 OF 9

9. Read the final guarantee

Run the program and use its theorem. They refer to the same execution, not two separately implemented algorithms.

LIBRARY CONSTRUCTION

Interface.correct combines the logical postcondition with the output observation and compiled-step bound. The companion's **sortTwo_value** specializes the observation. List lemmas give sortedness and permutation.

```
-- Guarantee shape (schematic)
postcondition observed in output
AND
steps ≤ preparation + overhead * work
```

STUDENT USE

Use **sortTwo_correct** for the functional result and **sortTwo_cost** for time. Their proofs are included in the checked companion. You can reuse these theorems without opening the adapter or the compiler.

```
#eval (sortTwo 9 4).value
-- [4, 9]
#eval (sortTwo 7 7).value
-- [7, 7]
```

RUNNABLE LEAN

```
example (x y : Nat) :
  (sortTwo x y).value.Pairwise (· ≤ ·) ∧
  (sortTwo x y).value.Perm [x, y] :=
  sortTwo_correct x y

example (x y : Nat) : (sortTwo x y).steps ≤ 155 :=
  sortTwo_cost x y
```

Component	Artifact handed to the next step
State + certified insertNext	Logical effect, precondition, and work.
twoBody + generated VCs	twoBody_correct: two insertions, $2r + 3$ credits.
twoInsertions + client	A reusable summary and client_correct.
Insertion.interface + compiler	sortTwo: value and steps, without fuel.
Interface.correct + list lemmas	Sortedness, permutation, and the 155-step bound.

The initial suffix is empty, giving an abstract bound of 3. The adapter supplies preparation 5 and overhead 50: $5 + 50 \times 3 = 155$. The companion derives this bound from Interface.correct.

Scope: sortTwo sorts exactly two values. Use Insertion.run for arbitrary lists and BFS.run for reachability; their theorems follow the same route.

10. FIND THE RIGHT KIND OF PROBLEM

When a proof stops working

Use the goal to decide whether a mathematical fact is missing or a routine step needs help.

Symptom	What to do next
"unknown identifier" or unknown tactic	Check the imports and namespace. Start from the complete companion file, not an isolated excerpt.
A precondition remains after <code>paper_steps</code>	Check the operation's logical requirements: <code>nonempty queue</code> , <code>nonempty todo</code> , or <code>valid neighbor</code> . Derive that fact from your invariant.
A credit inequality remains	First supply the charging fact, such as a fresh vertex leaving the potential. Then use <code>paper_credits</code> . A smaller budget may simply be false.
Sortedness or reachability remains	This is algorithm mathematics. Use a list/graph lemma or strengthen the invariant. Compiler tactics cannot supply the missing argument.
A long conjunction appears	Read each conjunct as a precondition, payment, or postcondition. The current interface still exposes some Lean bookkeeping.
Module docstring warning	Place a <code>/-! ... -/</code> module comment immediately after the imports. Do not suppress a real warning as an expected test result.

Three exercises

A. Change the source. Before running source 4, predict `seen`, `queue`, and the final output. Does a correct BFS mark every vertex?

B. Break a budget. In the local contract demo on page 6, replace the supplied budget by 0. Which obligation should become impossible? Restore the file.

C. Keep a duplicate. Run insertion sort on `[3, 1, 3, 2]`. What must the output be? Which part of the invariant rules out silently dropping one 3?

A useful learning sequence

First run an existing program. Next prove a local contract. Then complete an exit proof. Only then attempt a whole loop invariant. The framework currently assumes basic Lean proof skills; the companion and these exercises provide a starting scaffold.

11. CONTINUE WITHOUT GETTING LOST

Answers and a reading map

Use the smallest entry point that answers your current question.

Exercise answers

- A.** Initially seen = {4}, queue = [4], processed = {}. Vertex 4 has no neighbors. Processing it leaves queue empty and the output [4]. BFS is correct; the graph is disconnected.
- B.** The operation requires sorted.length + 1 credits, which is always positive. The generated payment obligation cannot fit that amount into a zero budget.
- C.** The output is [1, 2, 3, 3]. The permutation invariant preserves multiplicities. Sortedness alone would not rule out [1, 2, 3].

Clickable repository map

[Run and apply theorems](#)

Minimal examples with ordinary inputs and named results.

[BFS program and invariant proof](#)

The frontier argument and per-vertex charging scheme.

[Insertion-sort proof](#)

A small loop with a sortedness/permutation invariant.

[Public graph-operation contracts](#)

What calls require, change, and cost.

[Tutorial and API notes](#)

More detail on syntax, goals, and current limitations.

[Architecture map](#)

Optional: how maintainers connect the proof and execution layers.

Five words to remember

Invariant: a fact true at every loop header. **Potential:** a budget attached to a state. **Contract:** what a call requires and guarantees. **Frame:** what a call preserves. **VC:** an obligation sufficient to justify the next program fragment.

This guide is tied to the checked repository baseline **2c78e53**. The examples use the unit-cost natural-number RAM model. New data-structure implementations require library verification; general ownership inference and automatic invariant discovery are not provided.